

DATABASE ARCHITECTURE & PROCESS FLOWS

Sunrise School ERP — Database Essentials Explained

A Step-by-Step Plain-English Guide to the 68 Active PostgreSQL Tables & How Data Moves Behind the UI



16 End-to-End Processes Explained



68 Active Populated Tables



5 Core Design Principles



Live Transaction & Query Flows

A Plain-English Architectural & Process Guide to the 68 Live Database Tables

How data actually moves, changes, and stays secure behind every screen in the school ERP.

The 5 Golden Rules of the Database Architecture

Before looking at individual processes, five core design rules govern every single table in Sunrise ERP:

1 Every Table Belongs to a School (`school_id` Multi-Tenancy)

Sunrise ERP is built to be sold to multiple independent schools (tenants). It is not a single-school system with branches. Almost every table carries `school_id` derived from `TenantBase`. A student, invoice, or attendance record in Sunrise Public School can never be queried or seen by another school because every database query is scoped to `school_id`.

2 Lifetime Facts vs. Session Facts (`students` vs. `enrolments`)

A child has facts that belong to them for life (Name, Date of Birth, Admission Number, Blood Group). These live in `students`. But a child also has facts that belong to a single class and academic year (Roll Number, Class 10-A, Fee Invoices, Marks, Attendance). These hang off `enrolments`. This prevents last year's records from bleeding into next year when a child gets promoted.

3 Financial Immutability (Money is Never Edited)

In real accounting, you never change or delete an invoice or payment that was already committed. In this database:

- If an invoice is wrong, it is marked `status = 'void'` with an audited reason, and a new one is issued.
- If a payment bounced or was collected in error, it is reversed with a **contra payment entry** (a negative amount pointing at `reverses_payment_id`). The financial history is always 100% reconstructable.

4 A Balance is Always a Runtime `SUM()` , Never a Stored Column

There is no `outstanding_balance` column on `students` or `enrolments`. Storing a balance causes synchronization bugs whenever a payment is reversed, a discount approved, or an invoice line voided. Instead, what a family owes is computed on-the-fly by summing `(invoice_lines.amount - invoice_lines.discount) - payment_allocations.amount`.

5 Destructive Actions Require a Mandatory Typed Reason (`audit_log`)

Actions like voiding an invoice, reversing money, deleting a circular, or removing a student cannot be performed without typing a justification. The backend writes the user ID, timestamp, IP address, and before/after JSON states into `audit_log`.

PROCESS **PROCESS 1: Multi-Tenancy & School Onboarding** **THE REAL-LIFE GOAL**

When multiple schools use the software on the same cloud server, each school must have its own isolated data, branding, and academic session, with zero possibility of data leakage between schools.

 TABLES INVOLVED

- **schools**: The tenant record (School Name, Code, Affiliation Number, CBSE Board, City, State, Primary Color theme, Logo URL).
- **academic_years**: The school sessions (e.g. "2025-26", Start Date: 1 April, End Date: 31 March, Status: **active**).
- **settings**: School-specific configuration toggles (Sibling discount %, Late fee grace days, Timezone).

 HOW DATA MOVES STEP-BY-STEP

- 1 **School Registration:** A new school is created in **schools** with a unique **code** (e.g. 'SPS' for Sunrise Public School).
- 2 **Session Initialization:** An initial **academic_years** row is created linked to **school_id**.
- 3 **Default Settings:** Default configuration rows (timezone **Asia/Kolkata**, attendance threshold **75%**) are seeded into **settings**.
- 4 **All Subsequent Records:** Every student, teacher, fee plan, and bus route created from that day forward has its **school_id** automatically populated.
- 5 **Request Interception:** Whenever any API request arrives at the backend, the user's JWT token provides **user.school_id**. The database queries automatically append **WHERE school_id = :school_id**.

 THE CLEVER DESIGN RULE

Only three tables lack **school_id**:

- **schools** (it *is* the tenant),
- **permissions** (global system vocabulary),
- **scheduled_jobs** (backend worker tasks).

Every other table inherits from **TenantBase**, preventing developer oversight from causing cross-school data leaks.

PROCESS 2: User Authentication & Role-Based Access Control (RBAC) **THE REAL-LIFE GOAL**

Ensuring principals, accountants, teachers, and parents see only the screens and actions appropriate to their duties.

 TABLES INVOLVED

- **users** : Login credentials (`email` , `hashed_password` , `role`).
- **roles** : Named job roles (`Admin` , `Principal` , `Accountant` , `Teacher` , `Parent`).
- **permissions** : 146 fine-grained actions (e.g., `fees.invoice.read` , `exam.marks.lock`).
- **role_permissions** : Junction table linking roles to permissions.
- **user_roles** : Junction table linking users to roles.
- **employees** : Physical staff biodata linked to `users.id` .

 HOW DATA MOVES STEP-BY-STEP

1 **Login**: A user logs in with email and password at `/auth/login` .

2 **Password Verification**: The backend verifies the bcrypt password hash in `users` .

3 **Permission Resolution**: The backend queries:

`users` → `user_roles` → `roles` → `role_permissions` → `permissions`

4 **JWT Issuance**: A signed JWT token containing the `user_id` , `school_id` , and permitted action scopes is returned to the client.

5 **Screen Gating**: When an administrator opens the web app, the Screen Registry checks if the token holds the necessary read permissions (e.g., `fees.invoice.read`). If not, the menu item and route are cleanly hidden.

 THE CLEVER DESIGN RULE

Permissions are stored as atomic strings (e.g., `academics.class.read` , `fees.invoice.generate`). Separating `role_permissions` from `users` allows a school to create custom roles (e.g., "Senior Vice Principal") with tailored permissions without code changes.

PROCESS 3: Student Admission, Profiles & Guardians **THE REAL-LIFE GOAL**

Managing the journey of a child from an initial walk-in enquiry to an enrolled student assigned to a classroom, complete with parent contacts and emergency authorizations.

 TABLES INVOLVED

- **enquiries** : Initial parent phone call or front-desk visit.
- **students** : Permanent master record of the child.
- **guardians** : Parent and emergency contact directory.
- **student_guardian** : Many-to-many junction linking students to guardians.
- **enrolments** : Links the student to a specific classroom for an academic year.
- **class_sections** : The physical class (e.g., "Class 10-A").

 HOW DATA MOVES STEP-BY-STEP

[Enquiry at Front Desk]

| (converts to)

▼

[students Table] —(lifetime biodata: name, DOB, admission_no)

|

└─> [student_guardian Junction] → [guardians Table] (father, mother, phone)

|

▼

[enrolments Table] —(year-scoped: roll_no, class_section_id, academic_year_id)

|

▼

[class_sections Table] (Class 10-A, Room 201, Class Teacher)

- 1 **Front-Desk Enquiry:** An entry is logged in **enquiries** with the child's name, class of interest, and parent phone number.
- 2 **Student Record Creation:** When admitted, an official admission number is generated from **number_sequences** (e.g. 2024000001), and a row is inserted into **students**.
- 3 **Parent Linkage:** Father and mother records are created in **guardians**. Rows in **student_guardian** link them to the student, marking **is_primary = TRUE** for the SMS contact and **pickup_authorized = TRUE**.
- 4 **Classroom Enrolment:** A row is inserted into **enrolments** specifying **class_section_id** (Class 10-A), **academic_year_id** (2025-26), and **roll_no = 1**.

 THE CLEVER DESIGN RULE

guardians is separate from **students**:

- When two siblings study in the school, both **students** rows point to the **same** **guardians** row through **student_guardian**.
- This enables automatic sibling discount detection: the system checks if an incoming student shares parents with an already enrolled student and applies a 10% fee concession.

PROCESS 4: Classrooms, Subject Teachers & Timetable Scheduling **THE REAL-LIFE GOAL**

Organizing which teacher teaches which subject in which room, and generating a weekly conflict-free bell schedule.

 TABLES INVOLVED

- **class_sections** : Class 1-A through 10-A, assigned Room Number, and designated Class Teacher.
- **subjects** : Mathematics, English, Hindi, Science, Social Studies, etc.
- **class_subject_teacher** : Which teacher teaches which subject to which class section.
- **school_periods** : Daily period timetable (Period 1: 08:00–08:45, Break: 10:15–10:35).
- **timetable_slots** : The weekly schedule grid (Monday Period 1 = Mathematics in Class 10-A with Teacher TCH001).

 HOW DATA MOVES STEP-BY-STEP

- 1 **Section Allocation:** The administration assigns a Class Teacher to `class_sections.class_teacher_id`.
- 2 **Subject Allocation:** In `class_subject_teacher`, Teacher A is assigned to teach Science to Class 9-A for 6 periods/week.
- 3 **Slot Scheduling:** In `timetable_slots`, slots are assigned: Day, Period, Class Section, Subject, Teacher, Room.
- 4 **Conflict Checks:** PostgreSQL constraints automatically block illegal inputs:
 - A teacher cannot be in two classrooms during the same period (`UniqueConstraint("school_id", "day_of_week", "period_id", "teacher_id")`).
 - Two classes cannot use the same room simultaneously (`UniqueConstraint("school_id", "day_of_week", "period_id", "room_number")`).

PROCESS 5: Student Daily Attendance & Shortage Tracking **THE REAL-LIFE GOAL**

Allowing class teachers to take morning roll-call from the web or mobile app, while automatically calculating shortage warnings below CBSE's 75% threshold.

 TABLES INVOLVED

- **attendance** : Daily attendance records per student enrolment.
- **holidays** : Official calendar holidays excluded from working day counts.
- **settings** : Holds `attendance.shortage_threshold = 75`.

 HOW DATA MOVES STEP-BY-STEP

- 1 **Roll-Call Screen:** The teacher opens Class 10-A on today's date. The backend queries `enrolments` to populate the 40 students.
- 2 **Marking Attendance:** For each student, a row is inserted or updated in `attendance` :
 - `enrolment_id`, `date = '2026-09-14'`, `status = 'present'` (or `'absent'`, `'late'`).
- 3 **Upsert Safety:** If the teacher changes an entry from Present to Absent, `UniqueConstraint("school_id", "enrolment_id", "date")` updates the existing row instead of creating duplicates.
- 4 **Shortage Calculation:** When the office views attendance summary reports, the backend divides total present days by total working days (excluding `holidays`). Any student below 75% is highlighted on the Shortage Warning list.

PROCESS **PROCESS 6: The Complete Fee Lifecycle: Plans, Billing & Invoicing**

 THE REAL-LIFE GOAL

Configuring what each grade owes, generating monthly student bills with approved scholarships, and tracking invoices without error.

 TABLES INVOLVED

- `fee_heads` : Named fee categories (Tuition Fee, Development Charge, Transport Fee, Exam Fee).
- `fee_plans` : Master fee structure for a grade (e.g. "Class 10 Fee Plan 2025-26").
- `fee_plan_items` : The items within the plan: Head, Amount, Frequency (monthly, quarterly, annual).
- `fee_concessions` : Approved scholarships or sibling discounts.
- `fee_invoices` : The monthly invoice header.
- `fee_invoice_lines` : The itemized line charges on an invoice.

 HOW DATA MOVES STEP-BY-STEP

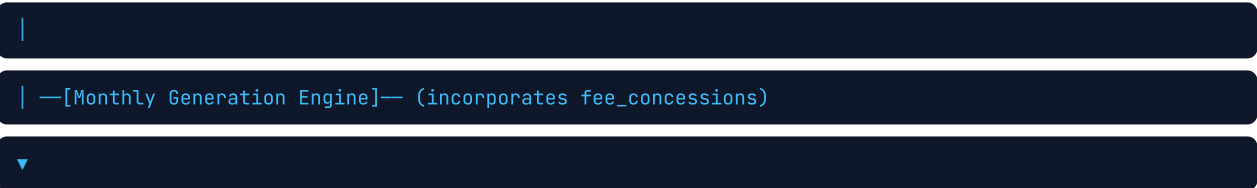
[`fee_heads`] (Tuition, Transport, Exam)



[`fee_plan_items`] (Tuition: ₹3,500/mo, Dev: ₹500/mo)



[`fee_plans`] ("Class 10 Fee Plan 2025-26")



[`fee_invoices`] (Header: Inv #INV-00142, Enrolment ID, Due Date: 10th)



[`fee_invoice_lines`] (Line 1: Tuition ₹3,500 [Discount ₹350], Line 2: Dev ₹500)

- 1 **Fee Setup:** A plan is configured in `fee_plans` and populated with `fee_plan_items`.
- 2 **Monthly Generation:** At the start of the month, the admin runs "Generate Invoices" for Class 10.
- 3 **Header Created:** A row is inserted into `fee_invoices`: `period_month = 9`, `period_year = 2026`, `due_date = '2026-09-10'`, `status = 'issued'`.
- 4 **Lines Created:** For each fee plan item, a row is inserted into `fee_invoice_lines`.
- 5 **Concession Application:** If an approved row exists in `fee_concessions` (e.g. 10% Sibling Concession on Tuition), the engine sets `fee_invoice_lines.amount = 3500.00` and `fee_invoice_lines.discount = 350.00`. Net payable is ₹3,150.00.

PROCESS 7: Fee Payments, Line-Level Allocation & Reversals

THE REAL-LIFE GOAL

Accepting parent fee payments at the counter or online, applying money accurately across bills, tracking overdue defaulters, and handling bounced payments.

TABLES INVOLVED

- `fee_payments`: Record of money received (Receipt No, Amount, Method: Cash/UPI/Cheque).
- `payment_allocations`: Connects payment money to specific `fee_invoice_lines`.
- `fee_invoices`: Updated from `issued` → `partially_paid` → `paid`.

HOW DATA MOVES STEP-BY-STEP

- 1 **Parent Pays:** A parent pays ₹4,000 via UPI.
- 2 **Payment Inserted:** A row is inserted into `fee_payments` with `amount = 4000.00`, `receipt_no = 'RCP-2025-0089'`, and an `idempotency_key` (to prevent accidental double-clicks).
- 3 **FIFO Allocation Engine:**
 - The backend finds all unpaid `fee_invoice_lines` for that student, ordered from oldest due date to newest.
 - It settles lines one by one.
 - For each settled line, a row is created in `payment_allocations` (`payment_id`, `invoice_line_id`, `amount`).
- 4 **Status Update:** If all lines on an invoice are fully paid, `fee_invoices.status` becomes `'paid'` and `settled_on = today`. If partially covered, it becomes `'partially_paid'`.
- 5 **Bounced Payment / Reversal:** If the payment bounced:
 - A **contra payment** is inserted into `fee_payments` with `amount = -4000.00` and `reverses_payment_id = original_id`.
 - Corresponding negative allocations are added to `payment_allocations`.
 - The invoice automatically reverts back to `'issued'` / unpaid.

WHY 'PAYMENT_ALLOCATIONS' POINTS TO LINES (NOT INVOICES)

Because accounting requires knowing *what* was paid: did the parent pay Tuition Fee or Transport Fee? By allocating money to individual invoice lines, the school's audit ledger maintains exact head-wise revenue recognition.

PROCESS 8: Examinations, Marks Entry, Paper Locks & CBSE Grading **THE REAL-LIFE GOAL**

Publishing datesheets, recording subject marks, locking papers against tampering, and generating CBSE report cards.

 TABLES INVOLVED

- **exams** : Exam series (e.g. "Half-Yearly Examination 2025-26").
- **exam_schedule** : The specific exam paper (Mathematics on 20 Sep, Max Marks: 80, Passing: 27).
- **marks** : Individual student score for a paper.
- **assessment_schemes** : CBSE assessment weights (Theory: 80%, Internal: 20%).
- **grading_scales** & **grade_bands** : Converts percentages into CBSE Letter Grades (A1, A2, B1, etc.).

 HOW DATA MOVES STEP-BY-STEP

- 1 **Exam Setup**: An exam is created in **exams**, and subject papers are scheduled in **exam_schedule**.
- 2 **Marks Entry**: Teachers enter scores on the marks grid. Rows are saved in **marks** (**marks_obtained** = 74.50, **enrolment_id**, **exam_schedule_id**).
- 3 **Paper Lock**: Once marks entry finishes, the coordinator clicks "Lock Paper". **exam_schedule.is_locked** = TRUE. Any future edit is rejected with **403 Forbidden** unless authorized with an audited override.
- 4 **CBSE Grading Calculation**: The engine sums components, computes percentage, and matches **grade_bands** (91-100% = A1, 81-90% = A2).
- 5 **Report Card Withholding Check**: Before rendering a report card, the backend checks if the student has unpaid fee invoices. If **exams.withhold_results_for_dues** = True and outstanding fees exist, the report card is automatically withheld.

PROCESS 9: School Transport Fleet & Seating Limits **THE REAL-LIFE GOAL**

Managing school buses, pickup/drop stops, and passenger lists while enforcing physical vehicle capacity limits.

 TABLES INVOLVED

- **vehicles** : Bus registration number, legal seating capacity, permit/fitness dates.
- **routes** : Bus circuit code, assigned Driver and Bus Attendant.
- **route_stops** : Ordered halts with pickup and drop times.
- **transport_fee_slabs** : Monthly fee based on distance (0-5 km, 5-10 km).
- **transport_assignments** : Student passenger allocations.

 HOW DATA MOVES STEP-BY-STEP

- 1 **Fleet Setup**: Bus **UP-32-AB-1234** is registered with **capacity** = 42.
- 2 **Route & Stops**: Route 1 is mapped with Stop 1 (07:15 AM), Stop 2 (07:25 AM). Each stop points to a **transport_fee_slabs** row.
- 3 **Student Assignment**: A child is assigned to Stop 2 in **transport_assignments**.
- 4 **Hard Capacity Enforcement**: Before committing, the database checks:

$\text{Active Passengers on Route} < \text{Vehicle Capacity (42)}$

If full, the allocation is rejected.

- 5 **Automated Billing**: During monthly fee generation, the active transport assignment automatically adds a Transport Fee line item to the student's invoice.

PROCESS **PROCESS 10: Classroom Supplies & Stock Inventory** **THE REAL-LIFE GOAL**

Tracking lab chemicals, equipment, and sports gear, allowing teachers to report low stock from their phone, and tracking admin restocking.

 TABLES INVOLVED

- `stock_items` : Inventory catalog (Item Name, Code, Category, Quantity on Hand, Minimum Alert Threshold).
- `stock_requests` : Requisitions and depletion flags.

 HOW DATA MOVES STEP-BY-STEP

- 1 **Catalog Setup:** Chemistry item "Copper Sulphate" is entered with `quantity_on_hand = 15` and `minimum_quantity = 5`.
- 2 **Teacher Mobile Flagging:** A chemistry teacher notices only 2 bottles remain. From the mobile app, the teacher flags low stock.
- 3 **Request Created:** A row is inserted into `stock_requests` (`status = 'pending'` , `request_type = 'depletion_flag'`).
- 4 **Admin Approval & Restock:** When new stock arrives, the admin approves the request. `stock_requests.status = 'approved'` and `stock_items.quantity_on_hand` increases to `25`.
- 5 **Low-Stock Alert:** Whenever `quantity_on_hand ≤ minimum_quantity`, the ERP dashboard highlights the item in red.

PROCESS **PROCESS 11: Grievances, Complaints & Helpdesk** **THE REAL-LIFE GOAL**

Providing parents and teachers a transparent channel to report issues (academics, transport, facilities), assigning tickets to staff, and recording resolutions.

 TABLES INVOLVED

- `grievances` : Ticket header (Ticket No, Submitter, Category, Priority, Status, Assigned Staff).
- `grievance_replies` : Chronological conversation thread.

 HOW DATA MOVES STEP-BY-STEP

- 1 **Submission:** A parent submits a complaint about the school bus delay via the mobile app.
- 2 **Ticket Generation:** A ticket is created in `grievances` (`ticket_no = 'GRV-2025-0012'` , `category = 'transport'` , `priority = 'high'` , `status = 'open'`).
- 3 **Staff Assignment:** The principal assigns the ticket to the Transport Manager (`grievances.assigned_to = employee_id`). Status moves to `'in_progress'`.
- 4 **Conversation Thread:** The transport manager replies explaining traffic conditions. The message is inserted into `grievance_replies`.
- 5 **Resolution:** Once solved, the ticket is marked `status = 'resolved'` with mandatory `resolution_notes`.

PROCESS **PROCESS 12: Security, File Storage & Audit Trails** **THE REAL-LIFE GOAL**

Maintaining a tamper-proof audit trail of administrative actions and managing uploaded documents (birth certificates, marksheets) securely.

 TABLES INVOLVED

- **audit_log** : Unalterable record of all sensitive operations.
- **document_types** : Master list of required documents (Aadhaar, Transfer Certificate, Birth Certificate).
- **documents** : Metadata for uploaded files (File URL, Size, MIME Type, Verification Status).
- **number_sequences** : Atomic sequence counter for human-readable IDs.

 HOW DATA MOVES STEP-BY-STEP

- 1 **Document Upload:** A birth certificate is uploaded for an admitted child. The physical PDF file is stored on disk/S3; a metadata row is inserted into **documents** referencing **document_types.id**.
- 2 **Verification:** The admissions officer verifies the certificate and clicks "Mark Verified". **documents.verified_by** = **user_id** and **verified_at** = **now()**.
- 3 **Audit Log Capture:** Whenever an invoice is voided, payment reversed, or notice deleted, the backend automatically writes a row to **audit_log** capturing:
 - Who did it (**user_id**),
 - What entity was changed (**entity_type** , **entity_id**),
 - The user's typed reason,
 - **before_state** and **after_state** JSON snapshots.

PROCESS **PROCESS 13: Teacher Recruitment, Candidate Intake & Onboarding** **THE REAL-LIFE GOAL**

Handling walk-in and online teacher candidate applications, generating official CBSE A4 formatted application dossiers, issuing job offers with salary terms, and automatically provisioning active employee credentials upon hiring.

 TABLES INVOLVED

- **candidates** : Candidate biodata, teaching qualifications, prior experience, post applied for, and application status (`applied` , `shortlisted` , `offered` , `hired` , `rejected`).
- **candidate_offers** : Formal job offer details (offered designation, department, monthly CTC salary, joining date).
- **employees** : Permanent staff record created automatically when candidate joining is confirmed.
- **users** : Login credentials provisioned for the new teacher (`TCH0xx` / `Teacher@123`).

 HOW DATA MOVES STEP-BY-STEP

- 1 **Candidate Intake:** Receptionist fills walk-in candidate details (`/recruitment`). A row is created in `candidates` (`application_no` = `'TR-2026-0012'`). Receptionist can print the official CBSE A4 printable form preview.
- 2 **Shortlisting:** School admin reviews qualifications and moves candidate status to `shortlisted` .
- 3 **Offer Letter:** Admin opens "Issue Job Offer" modal, sets designation ("TGT Mathematics"), department, salary (₹36,000/mo), and joining date. Row is inserted into `candidate_offers` . Status updates to `offered` .
- 4 **Onboarding Confirmation:** Admin clicks "Confirm Joining & Onboard". The system:
 - Creates a new `employees` record with unique code (e.g. `TCH013`),
 - Creates a new `users` account with teacher role,
 - Links `candidates.employee_id` = `employees.id` ,
 - Updates candidate status to `hired` ,
 - Displays teacher login credentials on screen for immediate handover.

 THE CLEVER DESIGN RULE

Strict RBAC separation: Receptionists can register candidates and print standard forms but are strictly forbidden from viewing or executing pipeline transitions (shortlist, offer, hire).

PROCESS 14: Teacher Leave Management & Timetable Substitution Gate **THE REAL-LIFE GOAL**

Allowing teachers to apply for personal leave via the mobile app, and requiring administrators to assign 100% timetable substitutions before leave can be approved, with zero mutation to the master timetable slots.

 TABLES INVOLVED

- **staff_leave_requests**: Teacher leave applications (dates, reason, status `applied` / `approved` / `rejected`, admin remarks).
- **substitutions**: Date-scoped proxy teaching duties covering affected periods.
- **timetable_slots**: The master weekly timetable (never edited).
- **staff_attendance**: Staff daily attendance register updated to `status = 'leave'` upon approval.
- **in_app_notifications**: Push alerts sent to applicant and assigned substitute teachers.

 HOW DATA MOVES STEP-BY-STEP

- 1 **Mobile Application:** Teacher applies for leave on the mobile app for a specific date range with a reason. A row is inserted into `staff_leave_requests` (`status = 'applied'`).
- 2 **Substitution Matrix Calculation:** Admin opens `/staff-leave` and inspects the substitution matrix. The backend queries `timetable_slots` for the teacher's scheduled periods on those dates and finds free, non-conflicting teachers.
- 3 **Provisional Substitutions:** Admin assigns available teachers to each period. Provisional rows are written to `substitutions` (`status = 'provisional'`).
- 4 **Strict Safety Gate:** The "Approve" button remains locked at 0% or partial coverage. Only when 100% of affected periods have assigned substitutes does the button turn green and unlock.
- 5 **Approval Commit:** Admin clicks "Approve Leave & Confirm Substitutions":
 - `substitutions.status` updates to `'confirmed'`,
 - `staff_leave_requests.status` updates to `'approved'`,
 - `staff_attendance` records are created with `status = 'leave'`,
 - `in_app_notifications` are dispatched to both the applicant and substitute teachers.
- 6 **Rejection & Clean Purge:** If the leave is rejected, all provisional substitutions for the request are immediately deleted (`DELETE FROM substitutions WHERE leave_request_id = :id`), leaving the timetable completely clean.

 THE CLEVER DESIGN RULE

Zero Mutation to Master Timetable: Master `timetable_slots` are never modified or overwritten. Substitutions are date-bound rows in `substitutions` that act as daily overlays. When the leave date passes, normal timetable operation resumes automatically without requiring restoration logic.

PROCESS 15: Session Rollover & Academic Promotion **THE REAL-LIFE GOAL**

Advancing an entire cohort of students from one academic session to the next (e.g. Class 9-A in 2025-26 to Class 10-A in 2026-27), reallocating roll numbers, and handling detentions or pass-outs cleanly.

 TABLES INVOLVED

- **academic_years** : Source session and target planning session.
- **enrolments** : Historical enrolments preserved; new session enrolment rows created.
- **class_sections** : Source and destination classroom sections.
- **audit_log** : Mandatory typed audit justification for the rollover operation.

 HOW DATA MOVES STEP-BY-STEP

- 1 **Wizard Step 1 (Target Selection):** Admin selects source session ("2025-26") and target session ("2026-27"), and chooses the class section to roll over.
- 2 **Wizard Step 2 (Roster Review):** The system displays the student list with proposed default outcomes (**Promote**). Admin can override individual students to **Detain** , **Pass Out** , or **Transfer Out** . Roll numbers are dynamically generated alphabetically.
- 3 **Wizard Step 3 (Safety Gate Confirmation):** Admin must type mandatory confirmation **PROMOTE** with a typed reason ("Annual academic progression after Final Term exams").
- 4 **Wizard Step 4 (Commitment):** The backend queries students, creates new **enrolments** rows for the target session and section, and logs the operation to **audit_log** .

 THE CLEVER DESIGN RULE

Historical preservation: Past enrolments, marks, attendance, and fee invoices are never overwritten. A student simply receives an additional **enrolments** record pointing to the new academic year and section.

PROCESS 16: Payroll Processing & Teacher Salary Disbursal **THE REAL-LIFE GOAL**

Calculating monthly salaries for teaching and administrative staff, factoring in allowances, statutory deductions (EPF, ESI, TDS), and attendance deductions, generating printable CBSE payslips and bank disbursal spreadsheets.

 TABLES INVOLVED

- **salary_components** : Catalog of earnings (Basic, DA, HRA, Special Allowance) and deductions (EPF, ESI, TDS, Professional Tax).
- **salary_structures** : Salary packages assigned to designations (PGT, TGT, PRT, Admin).
- **payroll_runs** : Monthly processing batch header (month, year, total gross, total deductions, total net payout).
- **payslips** : Individual employee payslip headers.
- **payslip_lines** : Itemized breakdown lines for each salary component on the payslip.

 HOW DATA MOVES STEP-BY-STEP

- 1 **Batch Initiation:** Bursar navigates to **/payroll** and clicks "Run Payroll" for the active month (e.g. September 2026).
- 2 **Computation Engine:** The backend retrieves all active employees, reads their assigned **salary_structures** , computes standard earnings, applies attendance deductions from **staff_attendance** , and calculates statutory deductions (EPF 12%, ESI 0.75%).
- 3 **Ledger Commit:** A **payroll_runs** record is created, and itemized **payslips** and **payslip_lines** are committed.
- 4 **Printable Payslip Generation:** Admin can open any staff member's payslip to render an official CBSE-compliant payslip with Lucknow school header, currency in words, and official signature blocks.
- 5 **Bank Transfer Export:** Admin exports **GET /admin/payroll/runs/{id}/bank-disbursal?format=csv** formatted for bulk NEFT/RTGS electronic bank disbursal.

Summary of Key Entities & Relationships

Area	Master Parent Table	Operational Child Table	Join Column
Multi-Tenancy	schools	All 65 tenant tables	school_id
Students	students (lifetime)	enrolments (session)	student_id
Guardians	students & guardians	student_guardian (M:N)	student_id , guardian_id
Academics	class_sections	timetable_slots	class_section_id
Substitutions	staff_leave_requests	substitutions	leave_request_id
Attendance	enrolments	attendance	enrolment_id
Staff Leave	employees	staff_leave_requests	employee_id
Recruitment	candidates	candidate_offers → employees	candidate_id , employee_id
Notifications	users	in_app_notifications	user_id
Fees Setup	fee_plans	fee_plan_items	fee_plan_id
Fee Billing	fee_invoices	fee_invoice_lines	invoice_id
Fee Realization	fee_invoice_lines & fee_payments	payment_allocations	invoice_line_id , payment_id
Exams	exams	exam_schedule → marks	exam_id , exam_schedule_id
Payroll	payroll_runs	payslips → payslip_lines	payroll_run_id , payslip_id
Transport	routes	route_stops → transport_assignments	route_id , route_stop_id
Inventory	stock_items	stock_requests	item_id
Grievances	grievances	grievance_replies	grievance_id
Audit	users	audit_log	user_id